

Maze Solver Robot

Ihfaz Rashid Sadat

ID - 23301499

Computer Science & Engineering
Brac University

ihfaz.rashid.sadat@g.bracu.ac.bd

Kaniz Yari Sanam

ID - 19101624

Computer Science & Engineering
Brac University

kaniz.yari.sanam@g.bracu.ac.bd

Nusfat Badroon Noor

ID - 22101242

Computer Science & Engineering
Brac University

nusfat.badroon.noor@g.bracu.ac.bd

Sahir Jawad Chowdhury

ID - 23301212

Computer Science & Engineering
Brac University

sahir.jawad.chowdhury@g.bracu.ac.bd

Umar Zawad Naqvi

ID - 24201433

Computer Science & Engineering
Brac University

umar.zawad.naqvi@g.bracu.ac.bd

I. ABSTRACT

This project presents the design and implementation of a maze solving robot that can find its way out of a maze by following a line on the ground. The robot is controlled by an Arduino Uno R3 SMD and uses three TCRT5000 infrared (IR) sensors to detect the path. Two DC motors, driven by an L298N motor driver, are used for movement. A simple and effective left-hand rule algorithm is implemented to make movement decisions. The goal of this project is to demonstrate sensor-based decision making and practical use of embedded systems to find its way out of the maze. **Keywords — Maze Solving Robot, Line Following, Arduino Uno, Infrared Sensors, Embedded Systems**

II. IMPLEMENTATION

A. Working Principle and Theory

The robot follows a line that represents the maze path. The TCRT5000 IR sensors work by emitting infrared light and measuring the reflected light from the surface. A dark surface absorbs more light, while a white surface reflects more light. Based on this difference, the sensors send digital signals to the Arduino.

The maze solving logic is based on the left-hand rule. According to this rule, the robot always tries to turn left first. If the left path is not available, it checks the right path, then the forward path. If no path is available, the robot stops. This rule ensures that the robot can eventually find an exit in a maze.

We selected the left-hand rule because it offers a simple and reliable maze-solving strategy that can be easily implemented using basic sensors and a microcontroller without requiring complex computations.

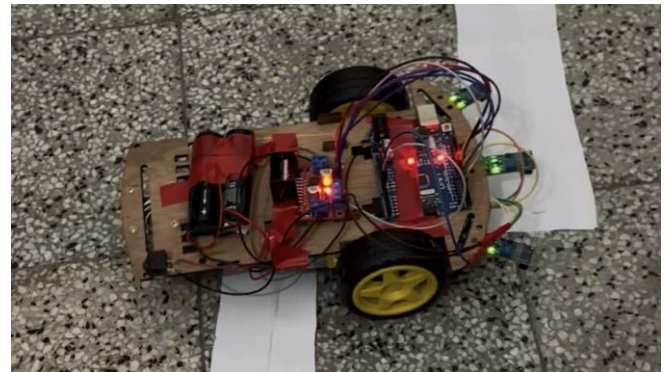
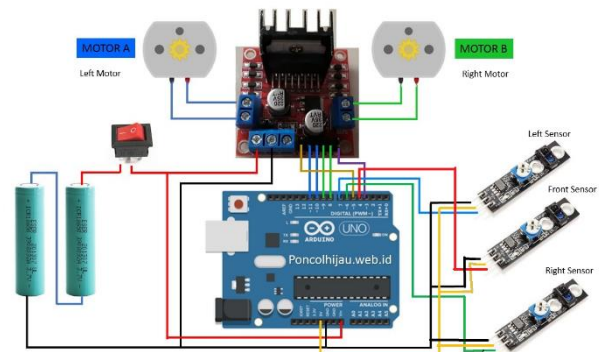
B. Circuit Diagram and Photos of Circuit Setup

The main hardware components used in this project are:

- Arduino Uno R3 SMD
- Three TCRT5000 IR sensors (Left, Middle, Right)
- L298N motor driver
- Two DC motors
- Wooden chassis

- Power supply (battery)

The IR sensors are connected to the digital input pins of the Arduino. The L298N motor driver is connected to the Arduino output pins to control motor direction and speed. The motor driver receives control signals from the Arduino and supplies sufficient current to drive the DC motors.



C. Arduino Code and its functionality

Arduino Code –

// MOTOR PINS

```
int mr1 = 8; // Right motor IN1
int mr2 = 9; // Right motor IN2
int ml1 = 10; // Left motor IN1
int ml2 = 11; // Left motor IN2
```

```
int enr = 3; // Right motor enable (PWM)
int enl = 5; // Left motor enable (PWM)
```

```

// SENSOR PINS
int sensorLeft = 7;
int sensorFront = 6;
int sensorRight = 4;

// SPEED SETTINGS
int forwardSpeed = 90;
int turnSpeed = 70;

// Adjust these experimentally
int turn90Delay = 250;

void setup() {
  pinMode(mr1, OUTPUT);
  pinMode(mr2, OUTPUT);
  pinMode(ml1, OUTPUT);
  pinMode(ml2, OUTPUT);

  pinMode(enr, OUTPUT);
  pinMode(enl, OUTPUT);

  pinMode(sensorLeft, INPUT);
  pinMode(sensorFront, INPUT);
  pinMode(sensorRight, INPUT);

  delay(3000); // startup delay
}

void loop() {
  int L = digitalRead(sensorLeft);
  int F = digitalRead(sensorFront);
  int R = digitalRead(sensorRight);

  // TRUTH TABLE IMPLEMENTATION
  if (L == 0 && F == 0 && R == 0) {
    turnLeft90(); // 000
  }
  else if (L == 0 && F == 0 && R == 1) {
    turnLeft90(); // 001
  }
  else if (L == 0 && F == 1 && R == 0) {
    turnLeft90(); // 010
  }
  else if (L == 0 && F == 1 && R == 1) {
    turnLeft90(); // 011
  }
  else if (L == 1 && F == 0 && R == 0) {
    turnRight90(); // 100
  }
  else if (L == 1 && F == 0 && R == 1) {
    moveForward(); // 101
  }
  else if (L == 1 && F == 1 && R == 0) {
    turnRight90(); // 110
  }
  else if (L == 1 && F == 1 && R == 1) {
    stopMotors(); // 111
  }
}

```

```

// MOVEMENT FUNCTIONS

void moveForward() {
  digitalWrite(mr1, HIGH);
  digitalWrite(mr2, LOW);
  digitalWrite(ml1, HIGH);
  digitalWrite(ml2, LOW);
  analogWrite(enr, forwardSpeed);
  analogWrite(enl, forwardSpeed);
}

void stopMotors() {
  analogWrite(enr, 0);
  analogWrite(enl, 0);
}

void turnRight() {
  digitalWrite(mr1, LOW);
  digitalWrite(mr2, HIGH);
  digitalWrite(ml1, HIGH);
  digitalWrite(ml2, LOW);
  analogWrite(enr, turnSpeed);
  analogWrite(enl, turnSpeed);
}

void turnLeft() {
  digitalWrite(mr1, HIGH);
  digitalWrite(mr2, LOW);
  digitalWrite(ml1, LOW);
  digitalWrite(ml2, HIGH);
  analogWrite(enr, turnSpeed);
  analogWrite(enl, turnSpeed);
}

void turnRight90() {
  turnRight();
  delay(turn90Delay);
  stopMotors();
}

void turnLeft90() {
  turnLeft();
  delay(turn90Delay);
  stopMotors();
}

```

Functionality of the code –

This Arduino based code implements a line following maze solving robot using a truth-table based decision algorithm. The robot is equipped with three infrared (IR) sensors - left, front, and right, and a differential drive system with two DC motors controlled via an L298N motor driver.

• Key Components and Pin Configuration

Motor Control Pins:

- Right motor: IN1 (8), IN2 (9), Enable (3 – PWM)
- Left motor: IN1 (10), IN2 (11), Enable (5 – PWM)

Sensor Pins:

- Left sensor: 7
- Front sensor: 6
- Right sensor: 4

Sensor Logic:

- 0 → Line detected (dark surface, non-reflective)
- 1 → No line detected (white surface, reflective)

- **Control Logic:**

The robot follows a truth table based on sensor inputs (L, F, R), where each sensor value is either 0 (line) or 1 (no line). The decision-making process is as follows:

L	F	R	Action
0	0	0	turnLeft90()
0	0	1	turnLeft90()
0	1	0	turnLeft90()
0	1	1	turnLeft90()
1	0	0	turnRight90()
1	0	1	moveForward()
1	1	0	turnRight90()
1	1	1	stopMotors()

- **Movement Functions:**

moveForward(): Both motors move forward at forwardSpeed (PWM = 90).

turnLeft() / turnRight(): Motors rotate in opposite directions at turnSpeed (PWM = 70).

turnLeft90() / turnRight90(): Execute a 90° turn using timed delays (turn90Delay = 250ms).

stopMotors(): Stops both motors by setting PWM to 0.

- **Initialization and Operation**

In setup(), all motor and sensor pins are configured, followed by a 3-second delay to allow placement of the robot.

The loop() continuously reads sensor values and executes the corresponding movement based on the truth table.

D. Challenges Faced & How We Solved Them

The first problem during this project occurred when we were using a 9V battery, although the circuit was powered on, the motors would not turn. We identified that the 9-volt battery was not supplying enough current to power the entire circuit along with the two motors simultaneously. While the battery could power a single motor individually and get it moving, once connected to the full circuit, the motors failed to operate.

To resolve this issue, we replaced the 9V battery with two 3.7V, (2600mAh batteries (model - ICR18650). After making this change, the motors began turning properly and the issue was resolved. The second problem arose when we implemented code from the website, Arduino Project Hub ^[1]. Upon running the program, the robot continuously spun counterclockwise (to the left). We later realized that the original code from the website ^[1], used a different logic convention, where 0 indicated no line detected and 1 indicated a line detected. In contrast, our circuit logic defined 0 as a line being detected and 1 as no line being detected. After making the necessary changes to the truth table of the code to match our logic, the robot began behaving as expected. The third problem we encountered involved sensor inaccuracies during turns. While the robot was turning, the sensors, particularly the one in the direction of movement, would go out of the line sometimes as the robot turns too much or too little, so it fails to detect the line or sometimes when turning right, the left sensor detects the line slightly, which was not supposed to happen and the robot tries to move to the left and vice versa. To counteract this issue, we reduced the turning delay from 350ms to 250ms. This adjustment resulted in slightly more accurate sensor readings, as there was less time for incorrect detections to occur, and consequently, the robot's turning behavior became a bit more reliable.

III. CONCLUSION

The maze solving robot was successfully designed and implemented. This project employed embedded systems, sensor integration, motor control, and algorithm implementation in a real-world scenario. The robot was able to follow the line and solve the maze implementing the left-hand rule algorithm with the help of the Arduino and the motor driver. In this project, we tried to implement a U-turn whenever the robot encountered a dead-end (not the actual exit of the maze) however it was not possible since there is no way of differentiating the actual exit of the maze and the dead-ends. If a U-turn were implemented whenever the robot detected a dead end, the robot would also perform a U-turn at the actual exit of the maze, causing it to attempt solving the maze again instead of stopping completely. As a result, the robot now stops whenever it encounters a dead-end or the actual exit of the maze. So differentiating between the dead-ends and the actual exit and being able to incorporate U-turns properly would be a great improvement to this project for the future.

REFERENCES

[1] Arduino Project Hub, "Line Following Robot," Arduino, Available: <https://projecthub.arduino.cc/lighththedreams/line-following-robot-34b1d3#section1>